Documentación técnica

TechStore

```JSON
{
  "equipo": [
    "Lucía Corral",
    "Pablo Demartini",
    "Carla Guisande",
    "Ignacio Hernández",
    "Ignacio Vidal"
  ],

  "docente": "Damián Wajser"
}
```

PP3: Desarrollo e implementación de sistemas en la nube

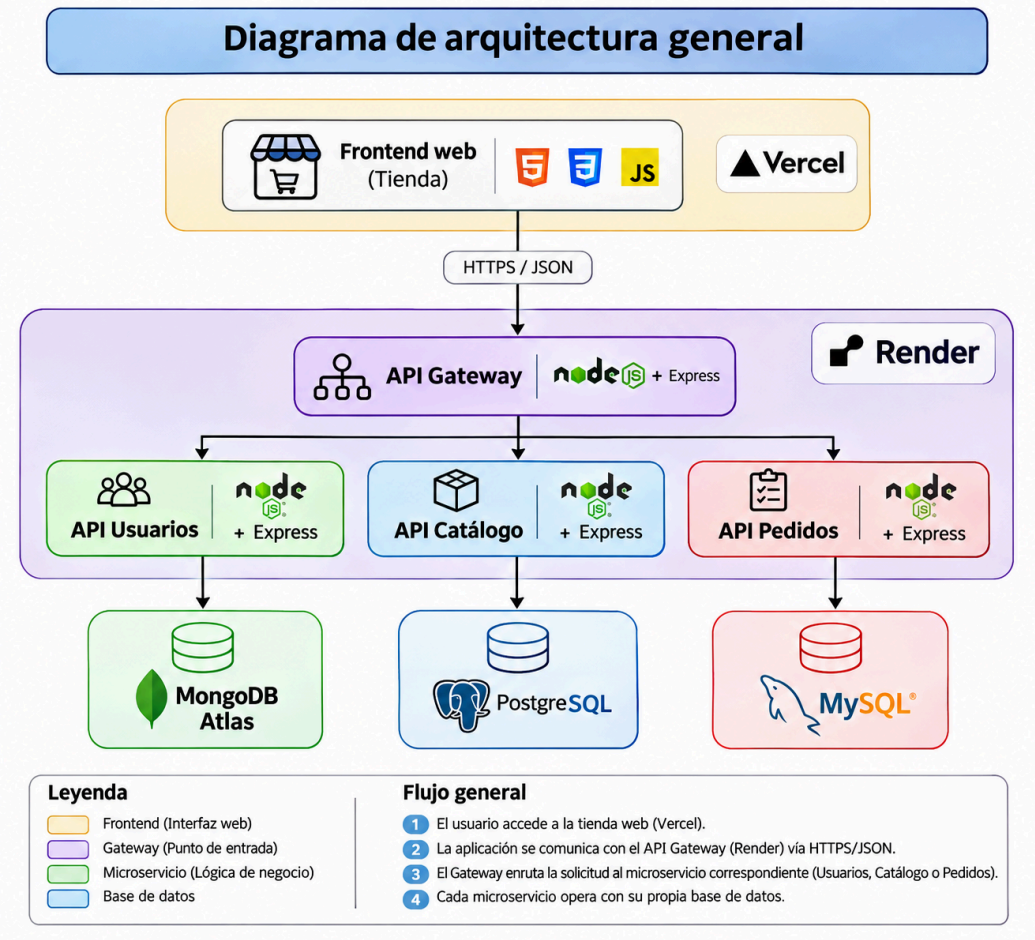
IFTS **N° 16**

-2026-

Índice

1. Diagramas de arquitectura 3
    1.1. Arquitectura general del sistema 3
    1.2. Capas de la aplicación 4
    1.3. Microarquitectura del servidor 5
        API de Catálogo 6
        API de Pedidos 7
        API Gateway 9
2. Diagramas de secuencia 10
    DS-01. Inicio de sesión 10
    DS-02. Consulta y actualización del perfil 11
    DS-03. Registro interno del pedido 12
    DS-04. Confirmación de compra 13
    DS-05. Registro de usuario 14
    DS-06. Consulta y búsqueda del catálogo 15
    DS-07. Gestión de carrito. 16
    DS-08. Consulta del historial. 17
3. Wireframes y mockups funcionales 18
        3.1. Pantallas principales 18
        3.2. Navegación e interacciones 18
        3.3. Wireframes y mockups funcionales. 20
            3.4. Cobertura funcional (Versión final) 24
        3.5 Mockups y flujo de navegación 26
4. Historias de usuario 27
    HU-01. Autenticación | DS-01 27
    HU-02. Consulta y actualización del perfil | DS-02 27
    HU-03. Generación de pedido | DS-03 27
    HU-04. Confirmación de la compra | DS-04 27
    HU-05. Registro | DS-05 28
    HU-06. Catálogo y búsqueda | DS-06 28
    HU-07. Carrito | DS-07 28
    HU-08. Historial de pedidos | DS-08 28
5. Evidencia de repositorios y MVP público 29

1. Diagramas de arquitectura



1.1. Arquitectura general del sistema

Se utiliza una arquitectura **cliente-servidor, organizada en tres capas y basada en microservicios**.Permitiendo separar las responsabilidades del sistema y distribuir sus funcionalidades entre componentes independientes.

- **Cliente-servidor**: el frontend funciona como cliente y proporciona la interfaz con la que interactúan los usuarios. A través de solicitudes HTTP, se comunica con los servicios backend, que procesan las operaciones y devuelven respuestas en formato JSON.
- **Arquitectura en tres capas**: el sistema separa la presentación (frontend), la lógica de negocio (APIs) y la persistencia de datos (bases de datos). Esta organización facilita el mantenimiento y permite modificar componentes sin afectar necesariamente al resto de la aplicación.

- **Microservicios**: las funcionalidades principales se distribuyen entre tres APIs independientes: Usuarios, Catálogo y Pedidos. Cada servicio se encarga de una parte específica del negocio y administra su propia base de datos, favoreciendo la modularidad y el despliegue independiente.

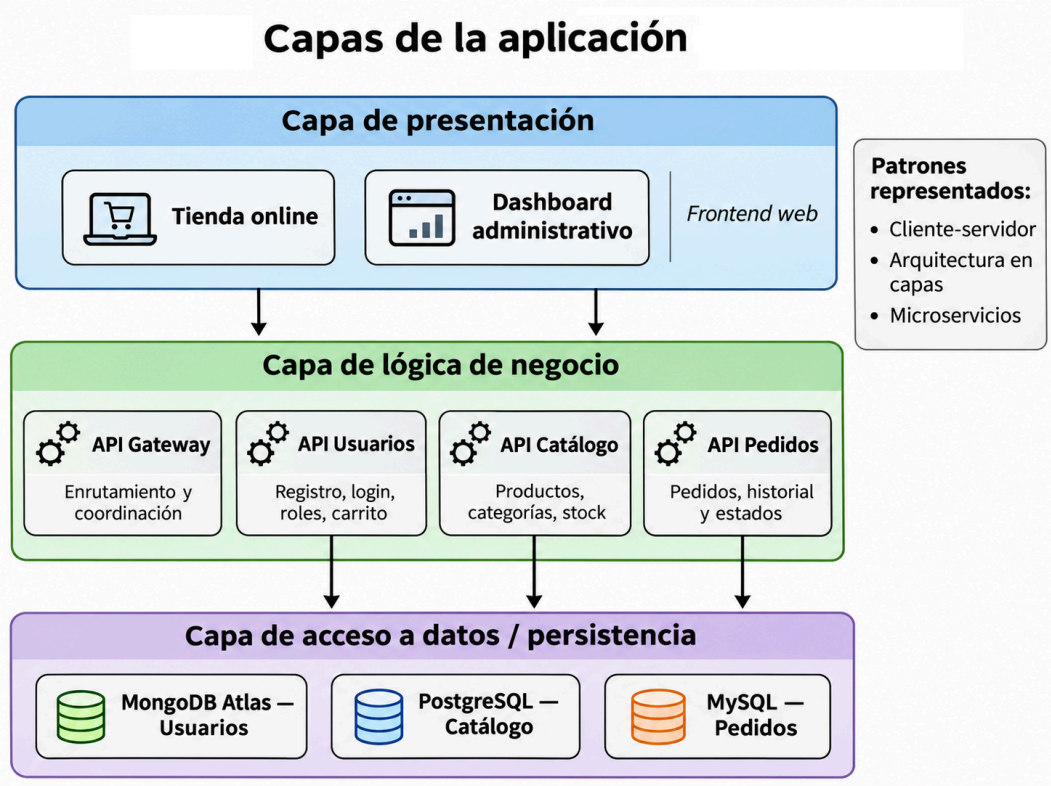
1.2. Capas de la aplicación

TechStore se organiza en tres capas, cada una con responsabilidades específicas:

Capa de presentación: corresponde al frontend web de Tech Store, orientado al cliente final. Permite consultar productos, gestionar el carrito, completar compras, consultar pedidos, realizar pagos y acceder a comprobantes. La versión actual no incluye un dashboard ni una interfaz administrativa.

Capa de lógica de negocio: está compuesta por tres microservicios independientes: API Usuarios (registro, autenticación, perfiles, roles internos y carrito), API Catálogo (productos, categorías, stock y cupones) y API Pedidos (gestión, historial, pagos y comprobantes). El API Gateway actúa como punto de entrada, recibe las solicitudes del frontend y las encamina hacia los servicios correspondientes. Algunas APIs incluyen capacidades administrativas a nivel backend, pero no se encuentran expuestas actualmente mediante una interfaz administrativa en el frontend.

Capa de acceso a datos: cada microservicio administra su propia base de datos: MongoDB Atlas para Usuarios, PostgreSQL para Catálogo y MySQL para Pedidos. Esta separación permite mantener la independencia de los datos y evitar el acceso directo de un servicio a la base de otro.



1.3. Microarquitectura del servidor

La arquitectura backend se compone de tres microservicios independientes y una API Gateway que centraliza la comunicación con el frontend.

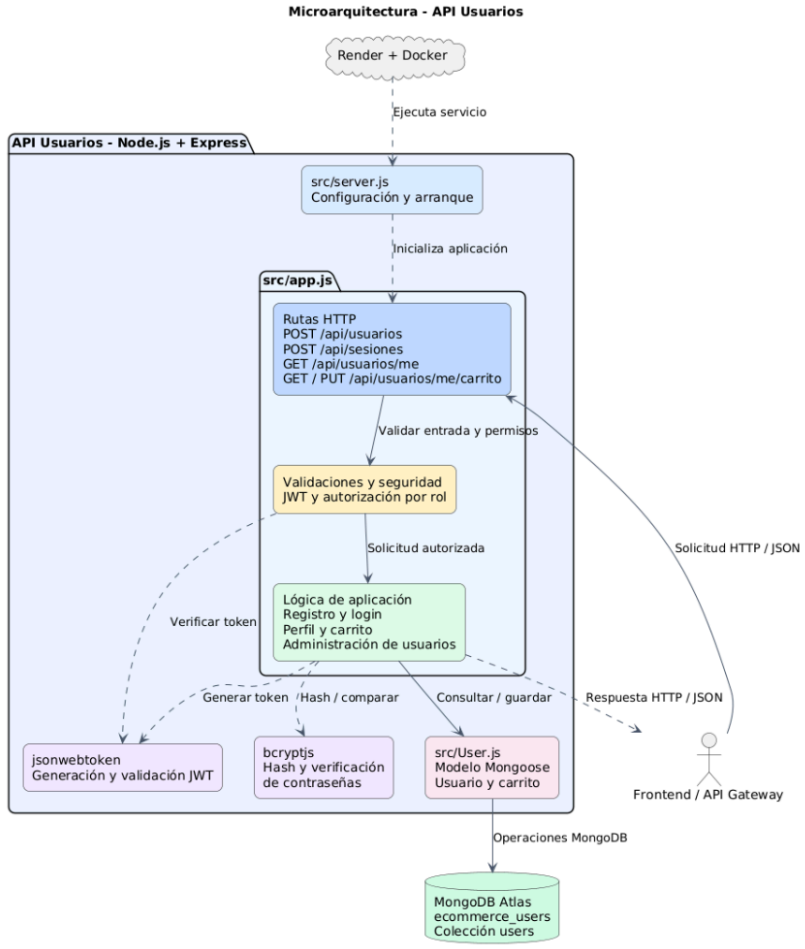
API de Usuarios

Gestiona el registro, la autenticación mediante JWT, los perfiles de usuario, los roles internos y la persistencia del carrito de compras. Aunque la API contempla operaciones administrativas, la versión actual del frontend no incluye un panel de administración.
Utiliza Express, Mongoose, bcryptjs y jsonwebtoken, con una base de datos MongoDB Atlas denominada ecommerce_users.
Su estructura principal comprende:

- src/server.js: configuración, conexión a MongoDB e inicio del servidor.
- src/app.js: definición de rutas, validaciones, autenticación y autorización.
- src/User.js: modelo de datos de usuarios y carrito mediante Mongoose.

Endpoints principales: POST /api/usuarios, POST /api/sesiones, GET /api/usuarios/me, GET/PUT /api/usuarios/me/carrito.
Repositorio: https://github.com/ignaci0vidal/pp3-api-usuarios
API: https://pp3-api-usuarios.onrender.com

Estado:https://pp3-api-usuarios.onrender.com/health



API de Catálogo

Administra los productos, categorías, precios, stock y cupones de descuento. Permite consultar el catálogo mediante búsquedas, filtros y paginación y gestiona los movimientos de inventario utilizados durante las compras y devoluciones. La API dispone además de operaciones administrativas internas, pero actualmente no existe una interfaz de administración en el frontend.

Utiliza Express y PostgreSQL, mediante la librería pg, e implementa mecanismos de autenticación y autorización.

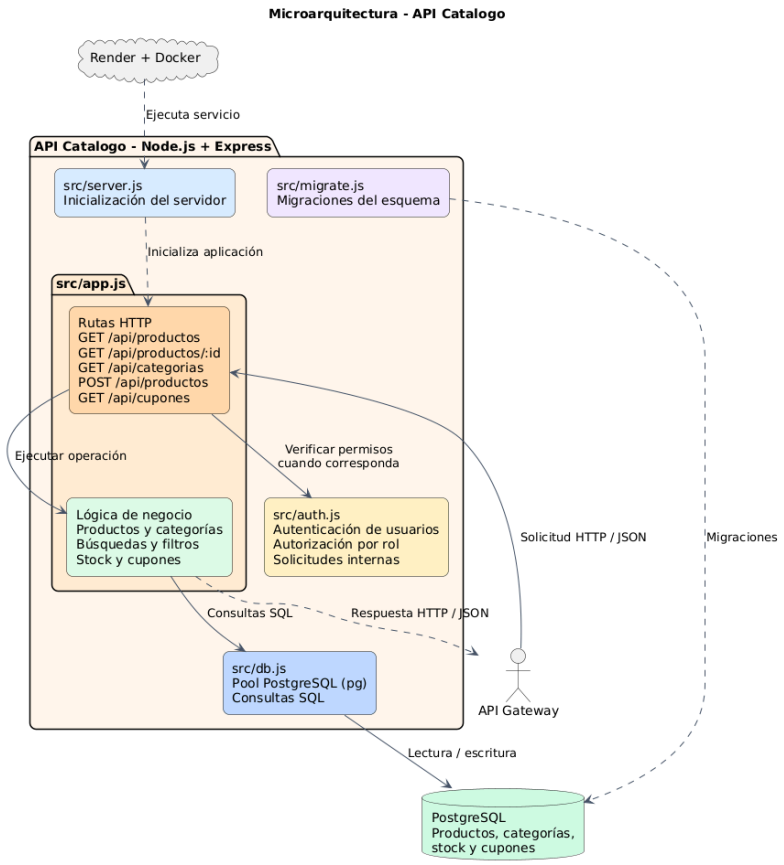
Su estructura principal comprende:

- src/server.js: inicialización del servidor.
- src/app.js: rutas HTTP, validaciones y lógica de las operaciones del catálogo.
- src/auth.js: autenticación y autorización.
- src/db.js: conexión a PostgreSQL.
- src/migrate.js: gestión de migraciones de la base de datos.

Endpoints principales: GET /api/productos, GET /api/productos/:id, GET /api/categorias, POST /api/productos, GET /api/cupones, GET /api/cupones/code/:code y POST /api/movimientos-stock (uso interno del Gateway para ventas y devoluciones de inventario).

Repositorio: https://github.com/Caaaaa2562/pp3-api-catalogo

API: https://pp3-api-catalogo.onrender.com
Estado:https://pp3-api-catalogo.onrender.com/health



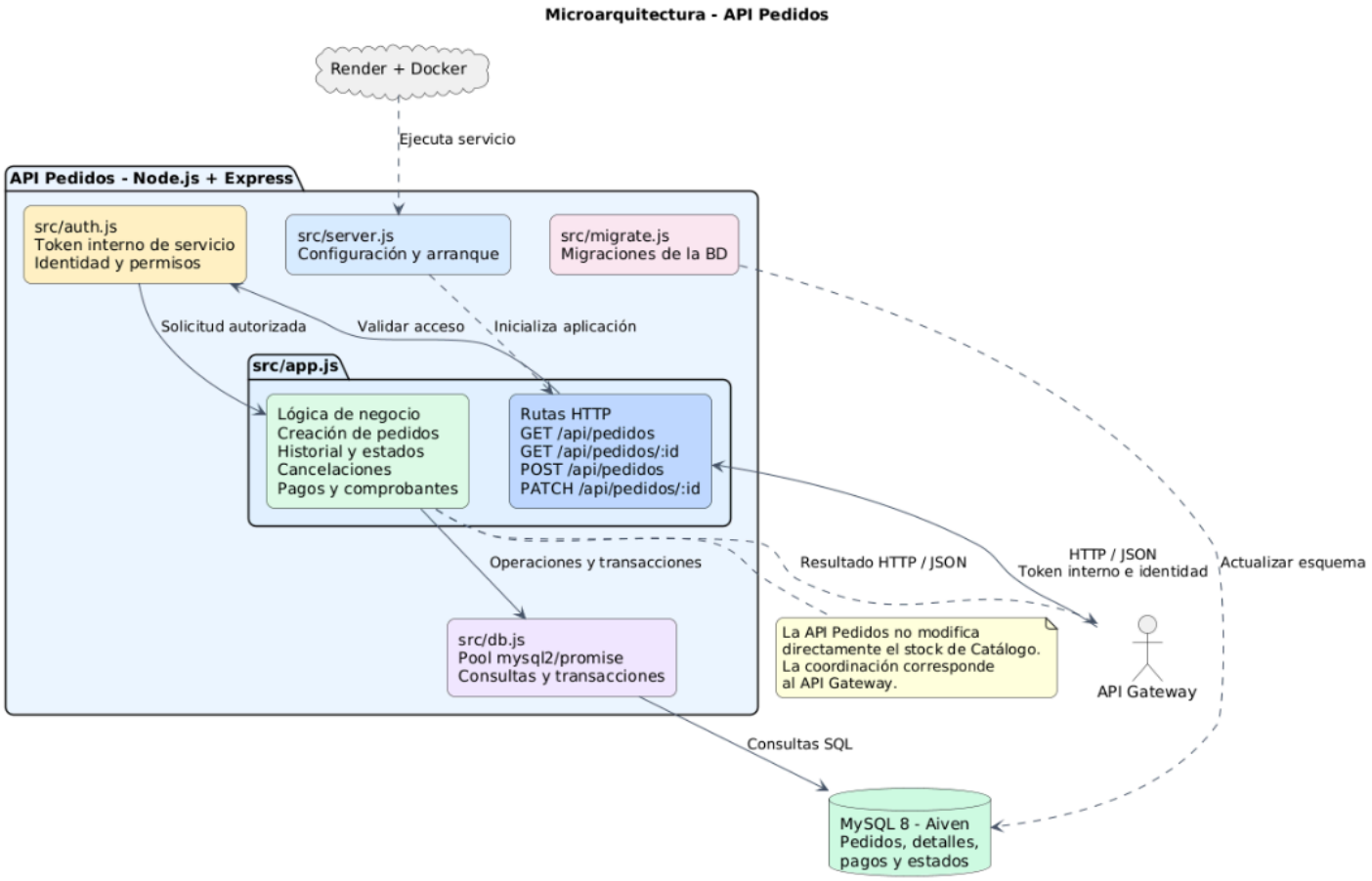
API de Pedidos

Gestiona el ciclo de vida de los pedidos, desde su creación hasta la confirmación, cancelación y entrega. También administra el historial de compras, los pagos simulados y los comprobantes. Utiliza Express y MySQL 8, con la librería mysql2, y una base de datos alojada en Aiven.
Su estructura principal comprende:

- src/server.js: inicialización del servicio.
- src/app.js: endpoints, validaciones y lógica de gestión de pedidos.
- src/auth.js: autenticación de solicitudes internas.
- src/db.js: conexión y administración del pool de MySQL.
- src/migrate.js: ejecución controlada de migraciones de la base de datos. Las migraciones aplicadas se registran en schema_migrations y no se vuelven a ejecutar en arranques posteriores.

Endpoints principales: GET /api/pedidos, GET /api/pedidos/:id, POST /api/pedidos, PATCH /api/pedidos/:id, POST /api/pedidos/:id/pagar, GET /api/pedidos/:id/comprobante y GET /api/pedidos/internal/intentos/:attemptId (uso interno para recuperar intentos).
Repositorio: https://github.com/ignaciohernandez/pp3-api-pedidos
API: https://pp3-api-pedidos-1.onrender.com

Estado: https://pp3-api-pedidos-1.onrender.com/health

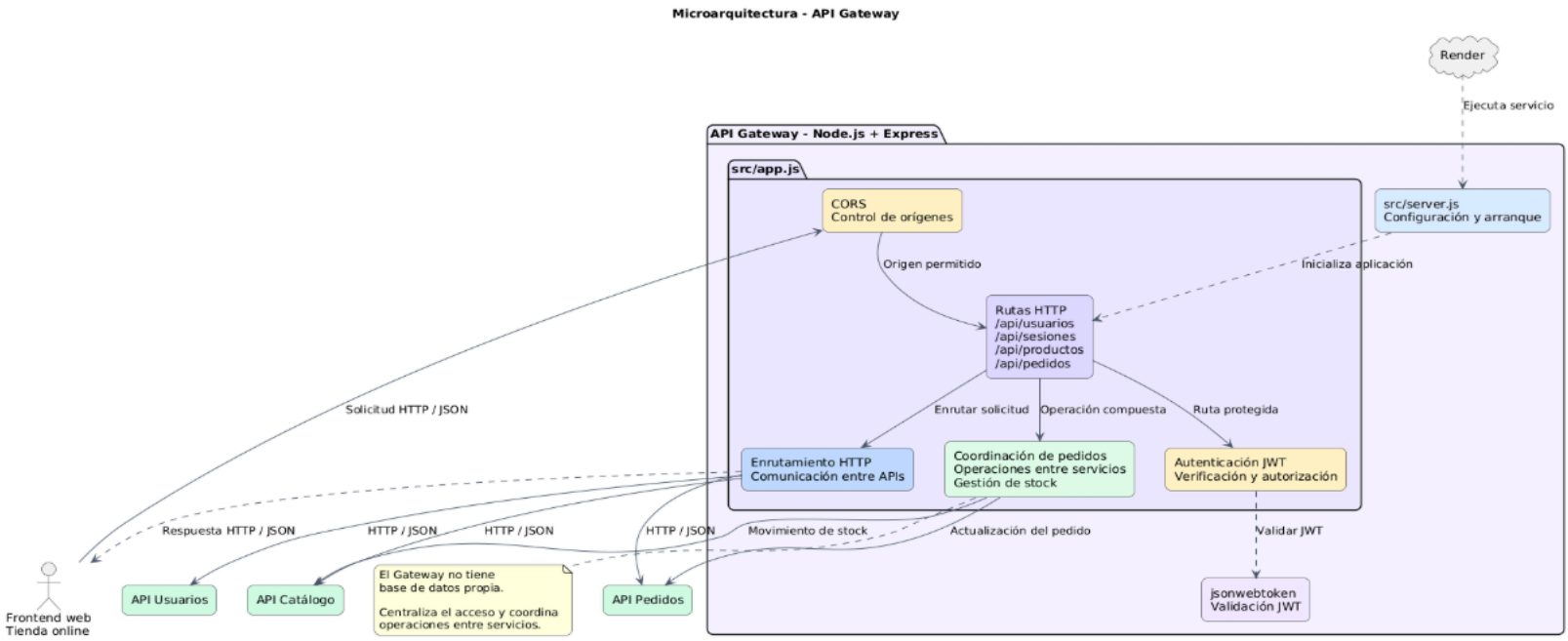


API Gateway

Actúa como punto de entrada del backend, centralizando las solicitudes del frontend y distribuyéndolas hacia las APIs de Usuarios, Catálogo y Pedidos.
Está desarrollado con Node.js y Express y utiliza cors y jsonwebtoken para controlar los orígenes permitidos y verificar la autenticación JWT. También coordina operaciones que requieren la participación de varios servicios, como la confirmación de pedidos y la gestión del stock.
Su estructura principal comprende:

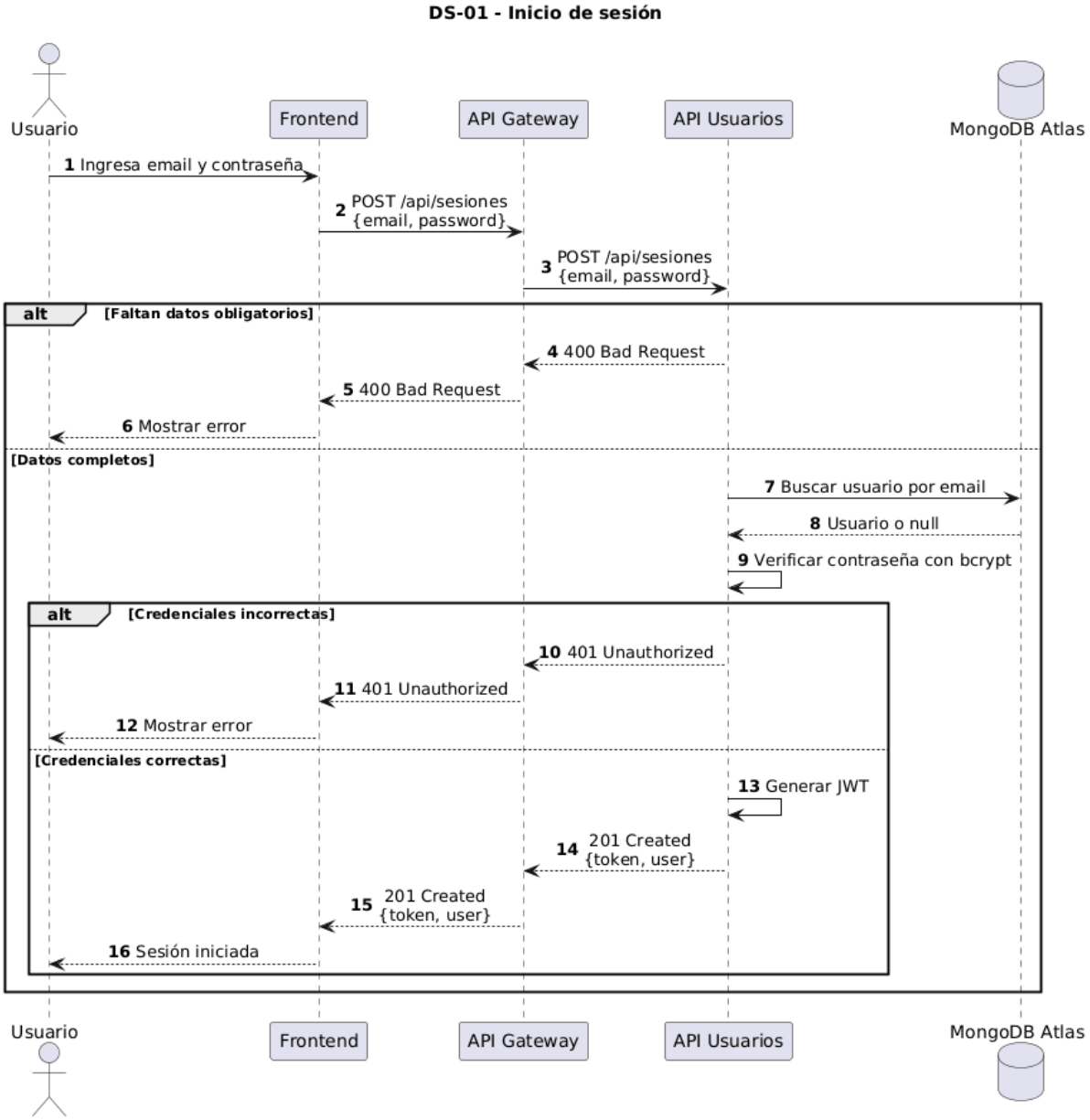
- src/server.js: configuración e inicio del servidor.
- src/app.js: definición de rutas, validación JWT, comunicación entre servicios y coordinación de operaciones.

A diferencia de los microservicios, no administra una base de datos propia. Se comunica con las APIs mediante HTTP y utiliza tokens internos para proteger determinadas operaciones entre servicios.
Endpoints de ejemplo: POST /api/sesiones, POST /api/usuarios, GET /api/productos, POST /api/pedidos, GET /health.
Repositorio: https://github.com/LuCorral/PP3-Ecommerce-Microservices
API: https://pp3-ecommerce-api-gateway.onrender.com/

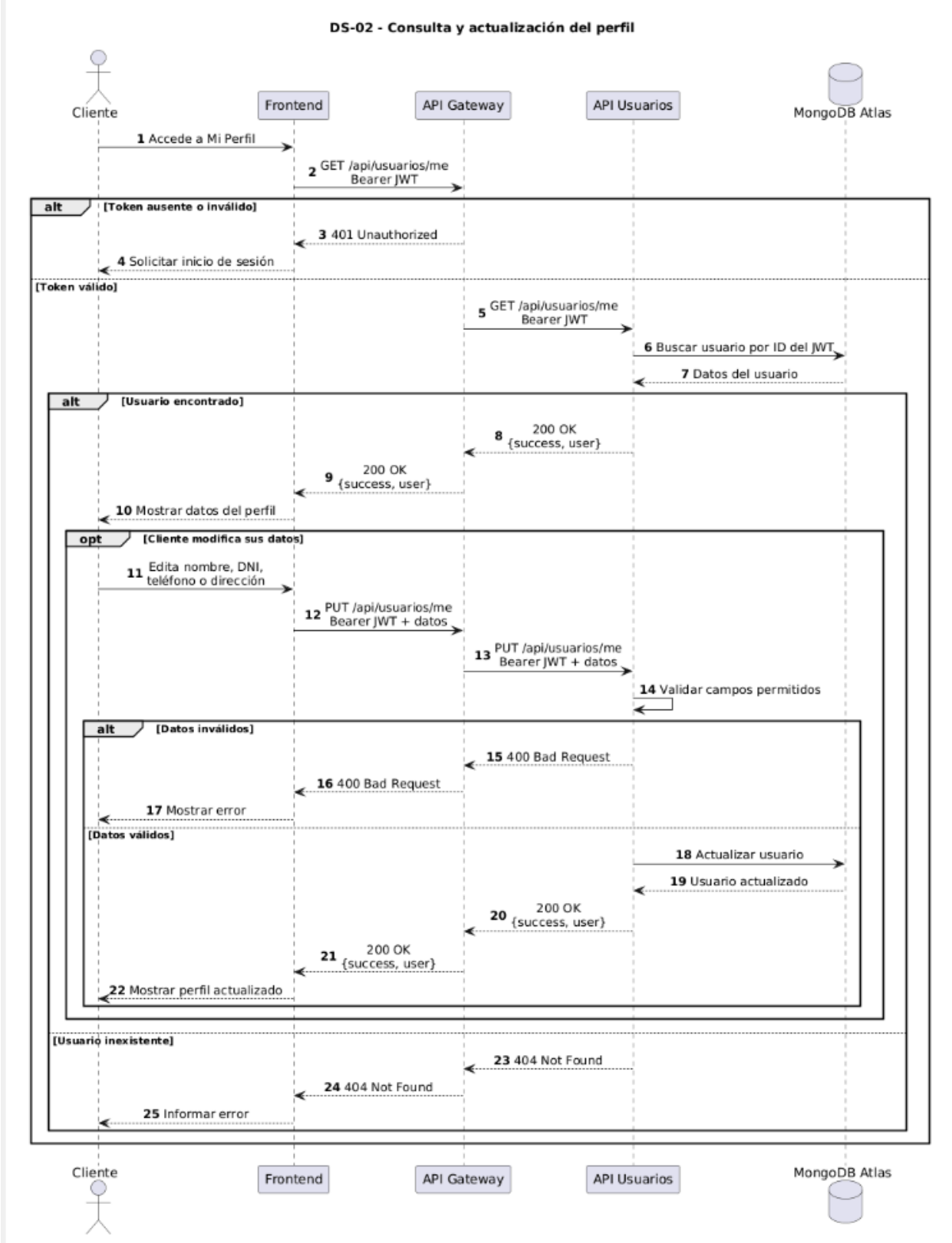


2. Diagramas de secuencia

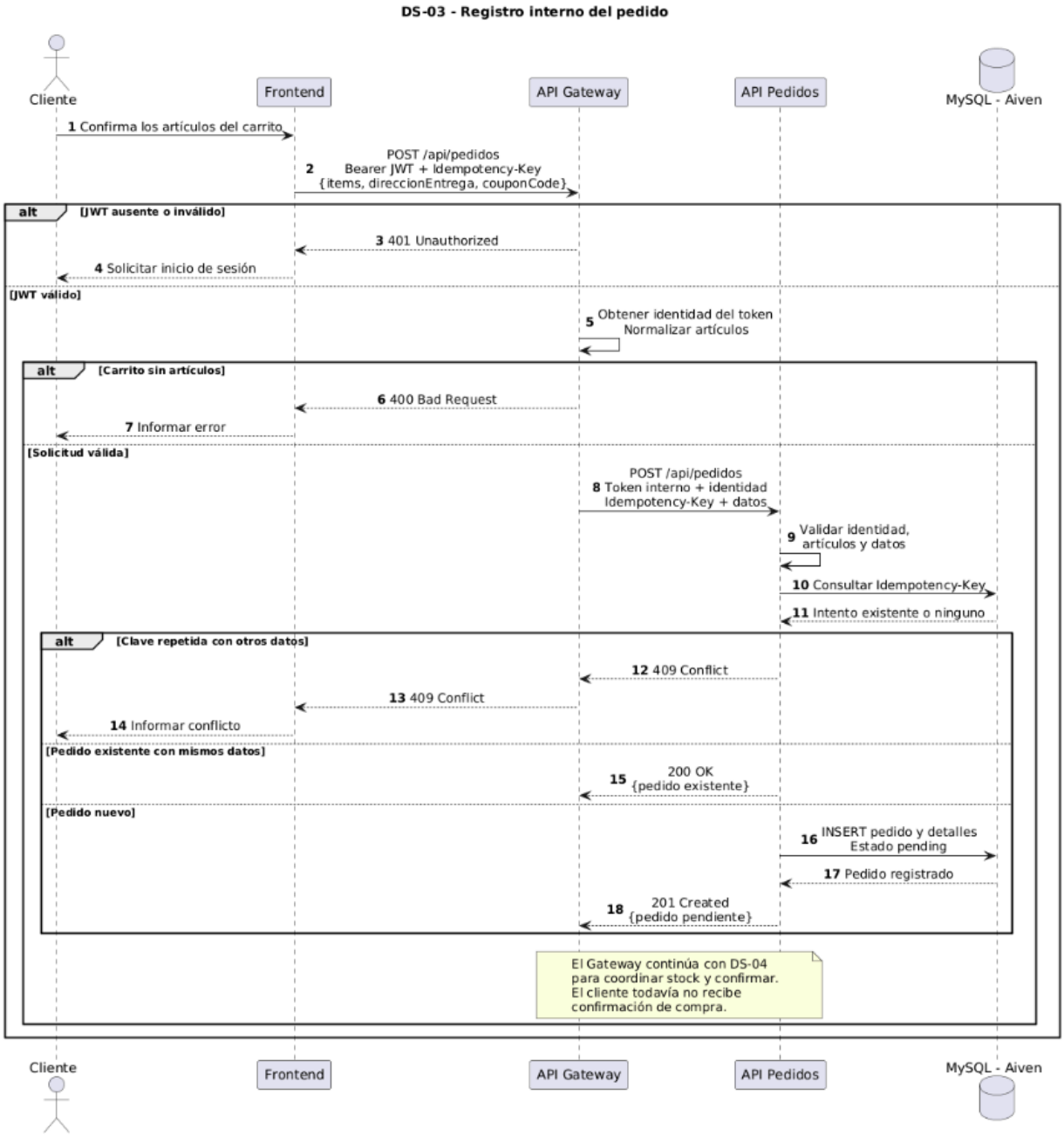
DS-01. Inicio de sesión



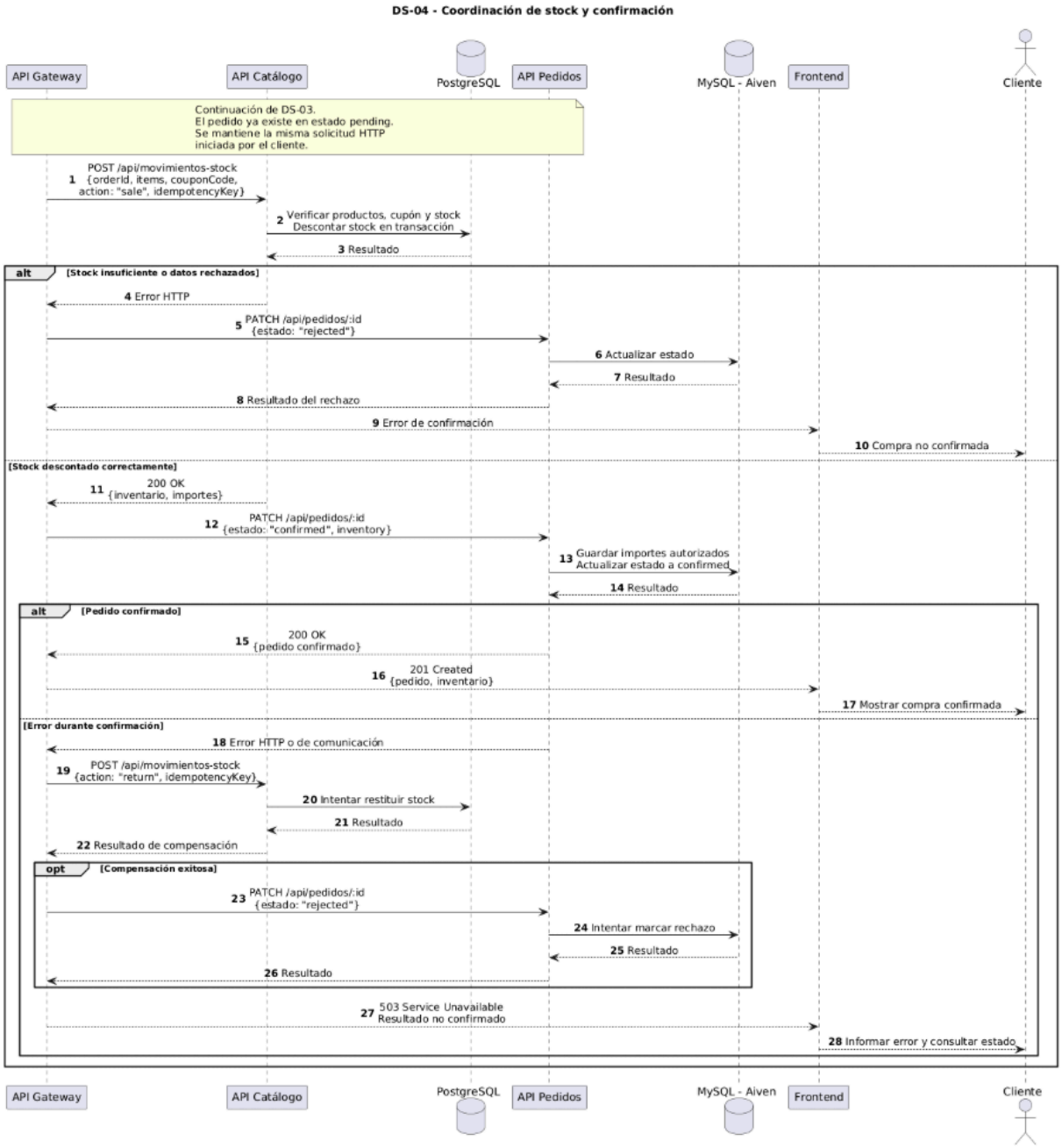
DS-02. Consulta y actualización del perfil



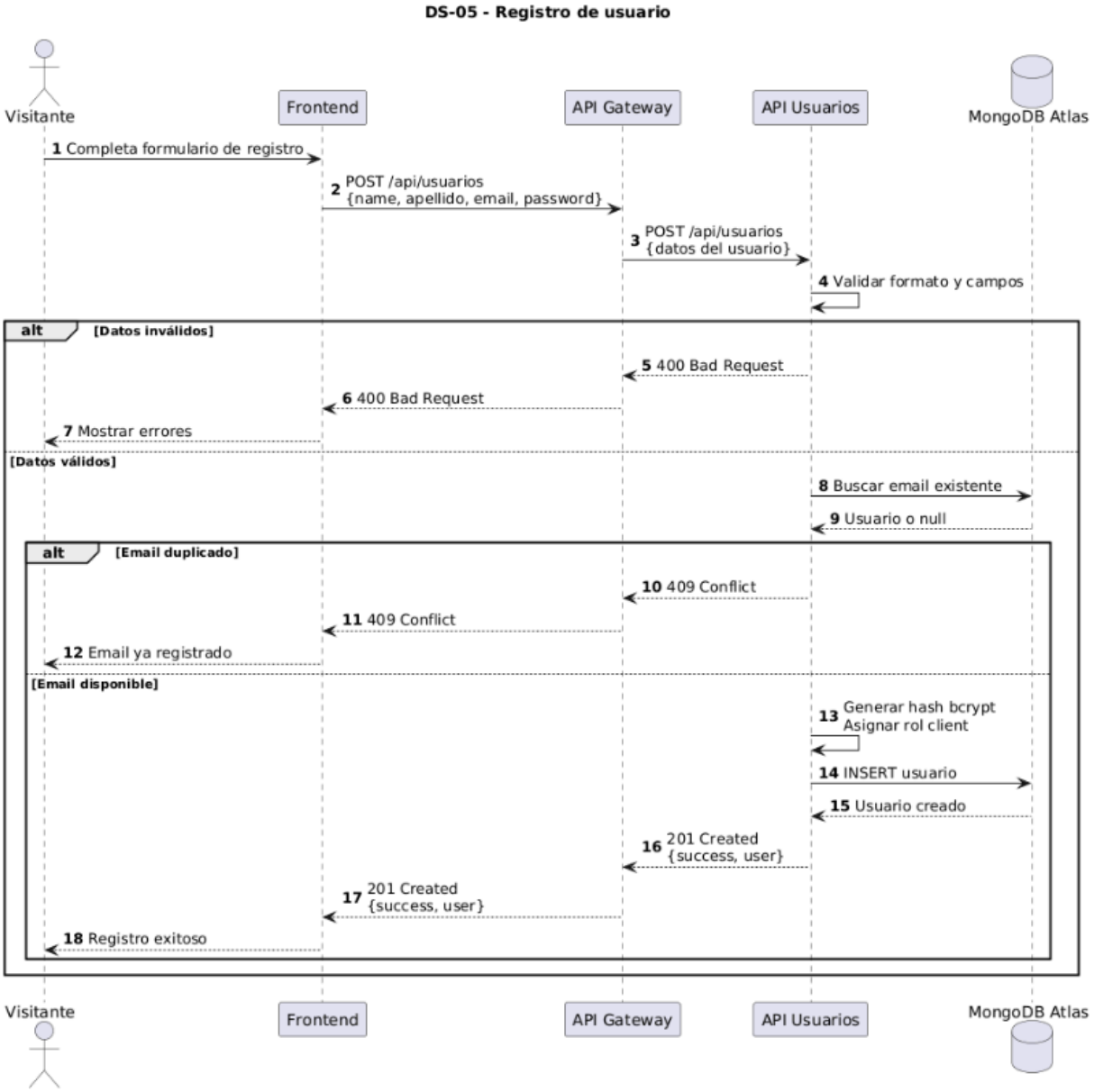
DS-03. Registro interno del pedido



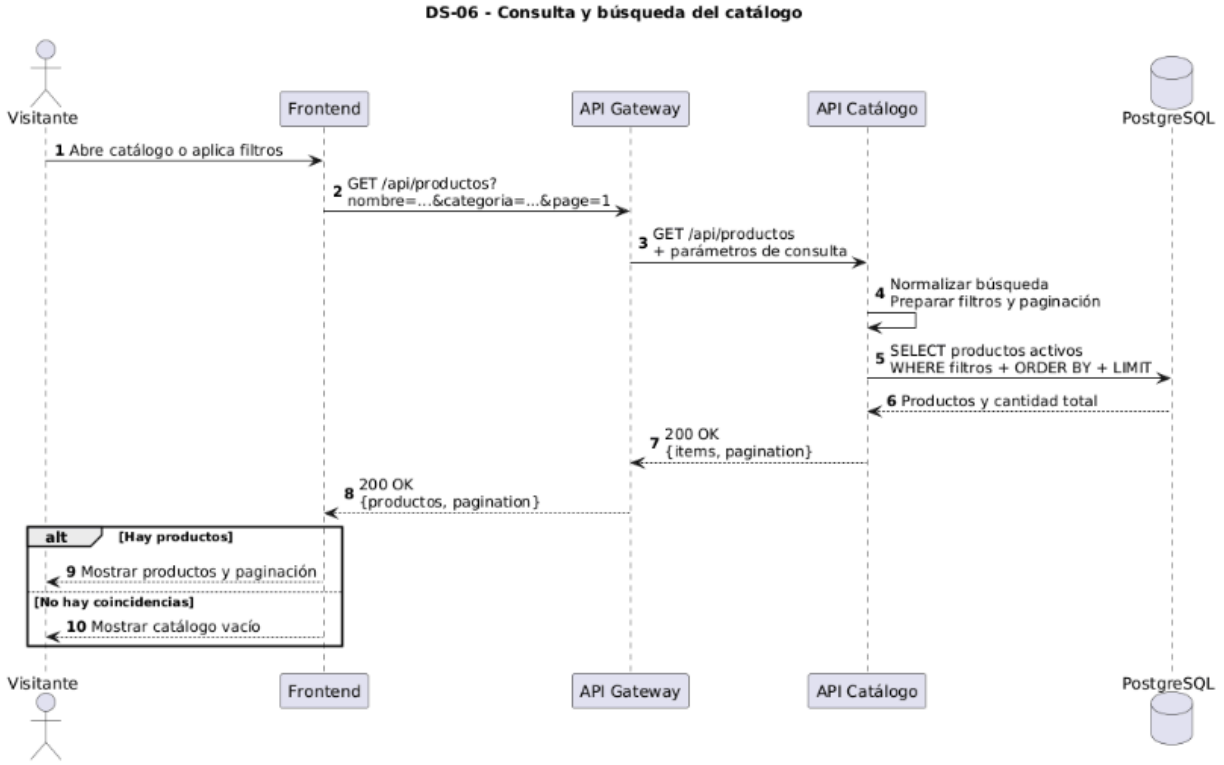
DS-04. Confirmación de compra



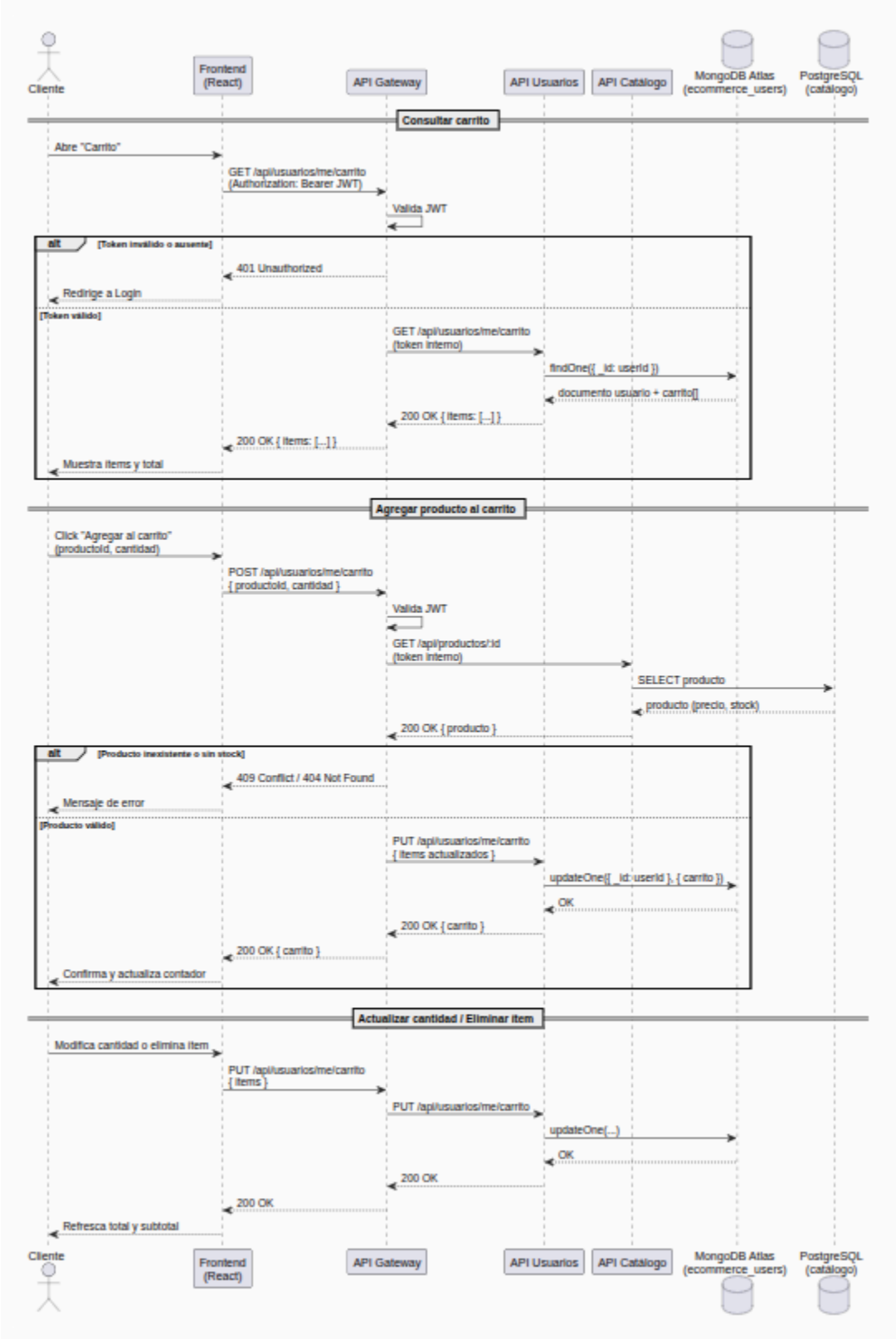
DS-05. Registro de usuario



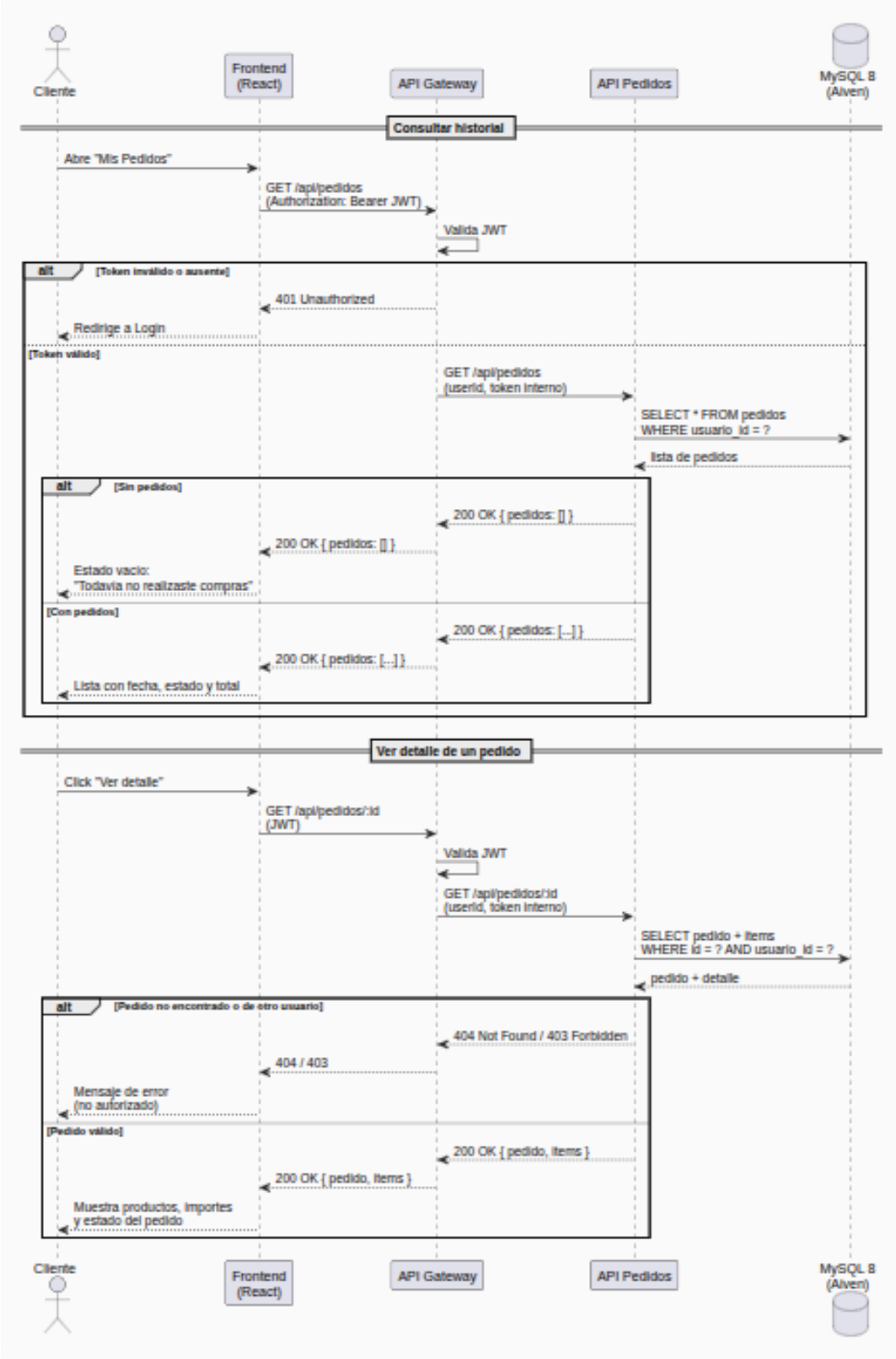
DS-06. Consulta y búsqueda del catálogo



DS-07. Gestión de carrito.



DS-08. Consulta del historial.



3. Wireframes y mockups funcionales

Los mockups funcionales de TechStore representan las principales pantallas de la aplicación y permiten visualizar su estructura, navegación y funcionalidades.

Para esta documentación se utilizan capturas del sistema desarrollado, mostrando las interacciones disponibles para los usuarios durante el proceso de compra.

3.1. Pantallas principales

Pantalla	**Funcionalidad**
Inicio / Catálogo	Visualización de productos disponibles.
Búsqueda y filtros	Búsqueda de productos por nombre y categoría.
Detalle de producto	Consulta de información, precio y disponibilidad.
Registro	Creación de una cuenta de usuario.
Inicio de sesión	Autenticación mediante correo electrónico y contraseña.
Carrito de compras	Gestión de productos y cantidades seleccionadas.
Confirmación de compra	Revisión de los datos y generación del pedido.
Historial de pedidos	Consulta de compras realizadas y sus estados.
Detalle de pedido	Visualización de productos, importes y estado de una compra.

3.2. Navegación e interacciones

La aplicación permite recorrer el catálogo, consultar productos y agregarlos al carrito. Para completar una compra, el usuario debe autenticarse, revisar su pedido y confirmar la operación.

Una vez registrada la compra, puede consultar su historial y acceder al detalle de los pedidos realizados.

Las siguientes capturas muestran las pantallas principales y su relación con los flujos representados en los diagramas de secuencia.

Inventario de pantallas MVP

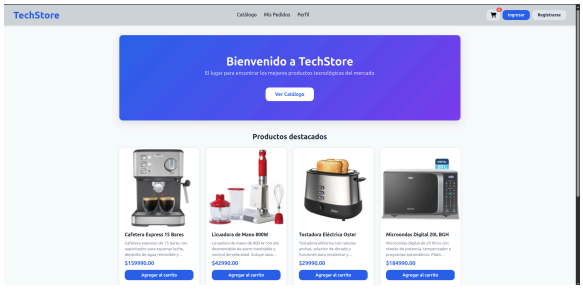
#	Pantalla	Ruta/ Sección	Funcionalidad	HU Asociada	DS Asociado
P-01	Inicio/Catálogo	/	Visualización de productos destacados y catálogo completo	HU-06	DS-06
P-02	Búsqueda y Filtros	/catálogo	Búsqueda por nombre y filtrado por categoría	HU-06	DS-06
P-03	Detalle de producto	/producto/:id	Consulta de información, precio y disponibilidad	HU-06	DS-06
P-04	Registro	/registro	Creación de cuenta de usuario	HU-05	DS-05
P-05	Inicio de sesión	/login	Autenticación con email y contraseña	HU-01	DS-01
P-06	Carrito de compras	/carrito	Gestión de productos y cantidades	HU-07	DS-07
P-07	Checkout/Confirmación	/checkout	Revisión de datos y generación del pedido	HU-03/04	DS-03/04
P-08	Mis pedidos (historial)	/pedidos	Consulta de compras realizadas y estados	HU-08	DS-08
P-09	Detalle de pedido	/pedidos/:id	Visualización de productos, importes y estados	HU-08	DS-08
P-10	Mi perfil	/perfil	Datos personales, DNI, teléfono y dirección	HU-02	DS-02

3.3. Wireframes y mockups funcionales.

3.3.1 Wireframes

Las siguientes imágenes corresponden a mockups funcionales y capturas de la versión implementada. Representan la estructura, navegación y comportamiento visible de las pantallas finales del MVP.

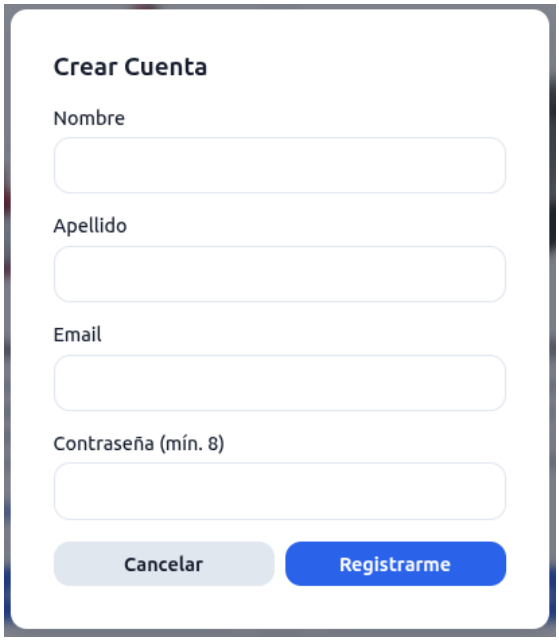
WF-01- Inicio/Catálogo



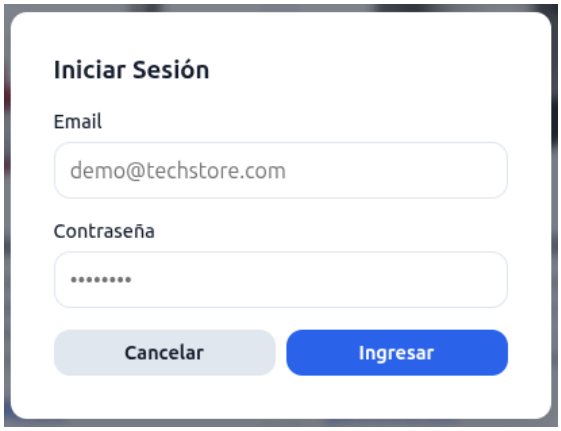
WF-02. Detalle de producto



WF-03. Registro



WF-04. Inicio de sesión



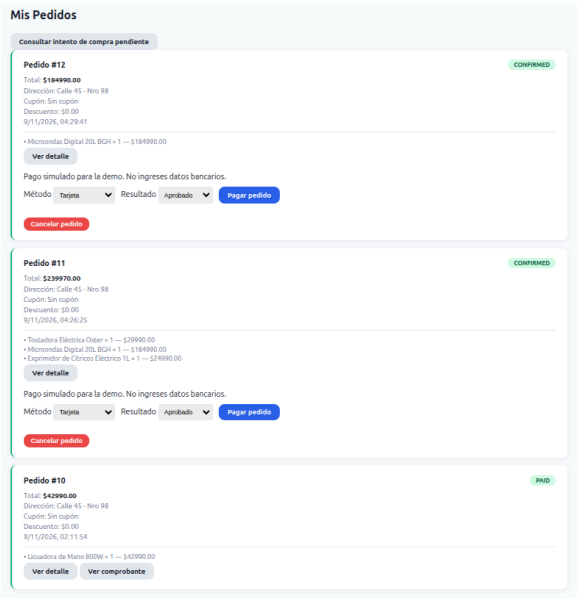
WF-05. Carrito de compras



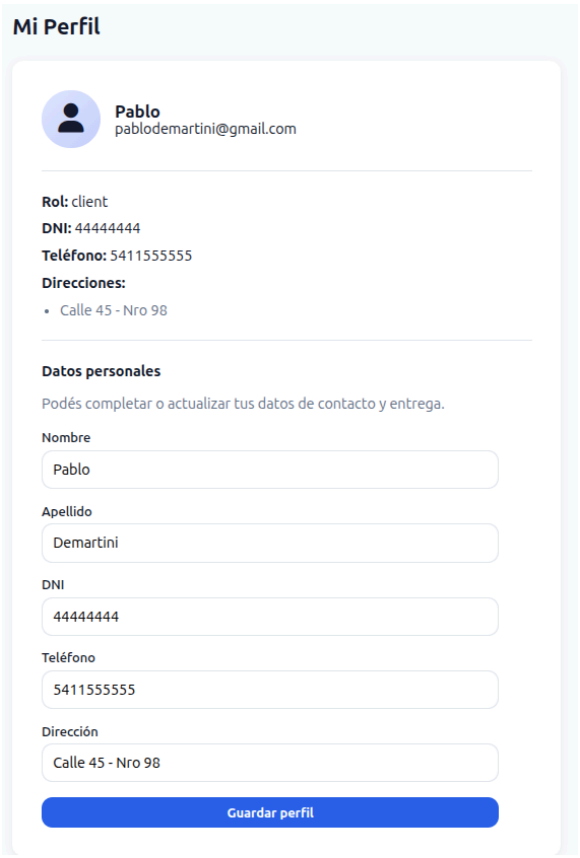
WF-06. Checkout / confirmación de compra



WF-07. Mis pedidos (historial)



WF-08. Mi perfil



3.4. Cobertura funcional (Versión final)

Los mockups se seleccionan para representar al menos el 80% de las pantallas y funcionalidades implementadas, conforme a los requisitos de la consigna. La cobertura se calcula sobre el inventario completo de pantallas de la versión final del MVP.

Fórmula: ***Cobertura = (pantallas representadas / pantallas implementadas) × 100***

Calculo de cobertura

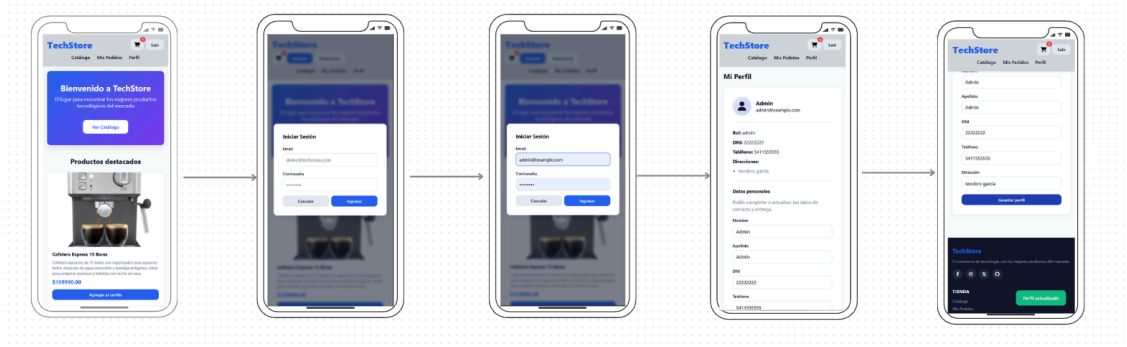
Aspecto	**Cantidad**
Pantallas implementadas en el MVP	10 (P-01 a P-10)
Pantallas representadas mediante mockups/capturas	10 pantallas funcionales agrupadas en 8 vistas principales; Búsqueda/Filtros se representa como variante del Catálogo y Detalle de Pedido como variante de Mis Pedidos.
Cobertura alcanzada	100% de las pantallas funcionales inventariadas, considerando las variantes indicadas.

Relación pantalla - historia de usuario - diagrama de secuencia

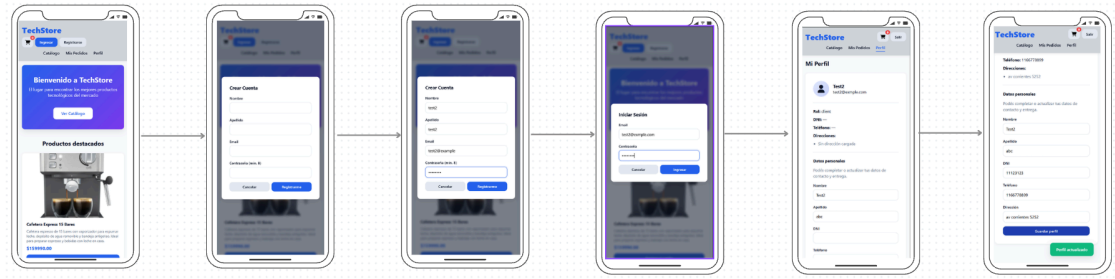
Pantalla	**HU**	**DS**
P-01 Inicio / Catálogo	HU-06	DS-06
P-02 Búsqueda y filtros	HU-06	DS-06
P-03 Detalle de producto	HU-06	DS-06
P-04 Registro	HU-05	DS-05
P-05 Inicio de sesión	HU-01	DS-01
P-06 Carrito	HU-07	DS-07
P-07 Checkout / Confirmación	HU-03 / HU-04	DS-03 / DS-04
P-08 Mis Pedidos	HU-08	DS-08
P-09 Detalle de pedido	HU-08	DS-08
P-10 Mi Perfil	HU-02	DS-02

3.5 Mockups y flujo de navegación

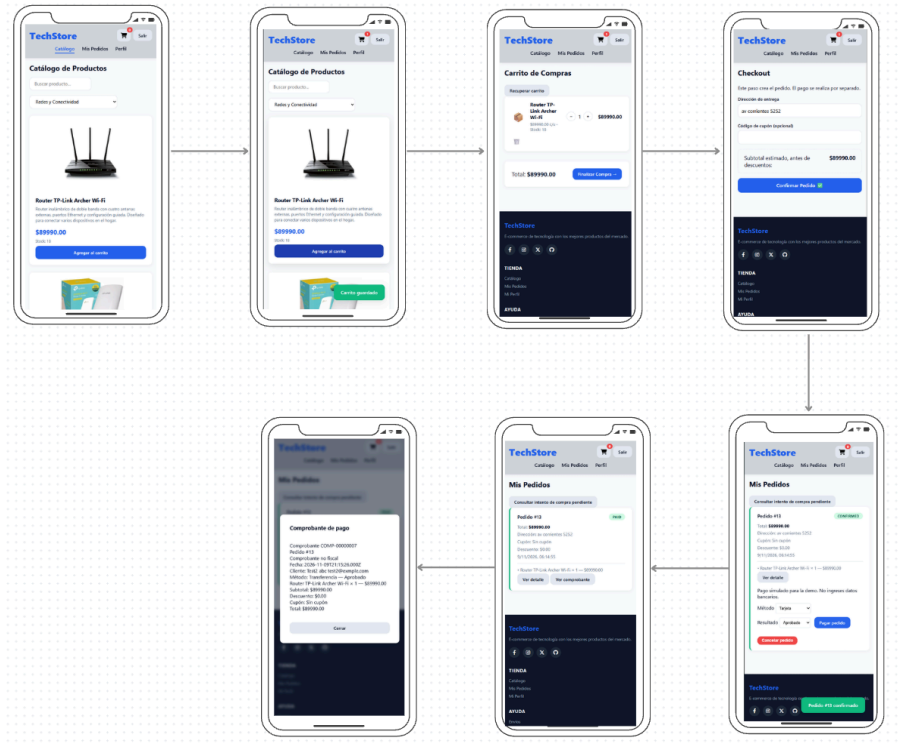
Inicio de sesión



Registro de usuario



Compra de producto y pago



4. Historias de usuario

HU-01. Autenticación | DS-01

Como usuario registrado,
quiero iniciar sesión
para acceder a mis funcionalidades personales.

Criterios: con credenciales válidas ingreso; con credenciales inválidas se informa el error; las rutas protegidas requieren autenticación.

HU-02. Consulta y actualización del perfil | DS-02

Como cliente autenticado,
quiero consultar y actualizar mis datos personales
para mantener mi información de contacto y entrega actualizada.

Criterios: El usuario autenticado puede consultar sus datos personales.Puede modificar los campos permitidos: nombre, apellido, DNI, teléfono y dirección.Los datos inválidos son rechazados y se informa el error.Los cambios válidos se guardan y se muestran actualizados.
Un usuario no puede modificar el perfil de otra cuenta.

HU-03. Generación de pedido | DS-03

Como cliente autenticado,
quiero generar un pedido con los productos de mi carrito
para iniciar el proceso de compra.

Criterios: El sistema verifica que el usuario esté autenticado. El pedido debe contener al menos un producto válido. Se registra inicialmente un pedido en estado pendiente. Se utiliza una clave de idempotencia para controlar solicitudes repetidas y evitar la creación de pedidos duplicados. El registro del pedido no implica todavía que la compra esté confirmada.

HU-04. Confirmación de la compra | DS-04

Como cliente autenticado,
quiero recibir la confirmación de mi pedido
para saber si la compra fue registrada correctamente y si los productos solicitados están disponibles.

Criterios: El sistema comprueba la disponibilidad de los productos antes de confirmar el pedido.
Cuando hay stock suficiente, se registra el movimiento de inventario y se confirma el pedido.Si el stock es insuficiente, el pedido no se confirma.Si ocurre un error después de descontar stock, el sistema intenta compensar la operación mediante su restitución.

El frontend informa el resultado y no presenta como confirmada una compra cuyo estado no pudo verificarse.

HU-05. Registro | DS-05

Como visitante,
quiero crear una cuenta
para realizar operaciones como cliente.

Criterios: los datos válidos crean un usuario; los inválidos se rechazan; no se admiten emails duplicados.

HU-06. Catálogo y búsqueda | DS-06

Como visitante,
quiero explorar y filtrar productos
para encontrar artículos de mi interés.

Criterios: se muestra el catálogo; los filtros modifican los resultados; si no hay coincidencias, se presenta un estado vacío comprensible.

HU-07. Carrito | DS-07

Como cliente,
quiero administrar los productos de mi carrito
para preparar el pedido.

Criterios: puedo consultar los ítems; se actualizan cantidades/contenido según las operaciones admitidas; el total mostrado resulta consistente con los datos.

HU-08. Historial de pedidos | DS-08

Como cliente autenticado,
quiero consultar mis pedidos anteriores
para conocer su información y estado.

Criterios: solo accedo a los pedidos autorizados; se muestra el estado; se maneja correctamente el historial vacío.

5. Evidencia de repositorios y MVP público

PP3 - FRONTEND
https://pp3-frontend-alternativo.vercel.app/
Repositorio: https://github.com/ignaci0vidal/pp3-frontend-alternativo
GitDiagram: https://gitdiagram.com/ignaci0vidal/pp3-frontend-alternativo

PP3 - API GATEWAY
https://pp3-ecommerce-api-gateway.onrender.com/
Repositorio: https://github.com/LuCorral/PP3-Ecommerce-Microservices
GitDiagram: https://gitdiagram.com/lucorral/pp3-ecommerce-microservices

PP3 - API de Usuarios
Repositorio : https://github.com/ignaci0vidal/pp3-api-usuarios
API : https://pp3-api-usuarios.onrender.com
Estado del servicio: https://pp3-api-usuarios.onrender.com/health
GitDiagram: https://gitdiagram.com/ignaci0vidal/pp3-api-usuarios

PP3 - API de Catálogo
Repositorio : https://github.com/Caaaaa2562/pp3-api-catalogo
API : https://pp3-api-catalogo.onrender.com
Estado del servicio: https://pp3-api-catalogo.onrender.com/health
GitDiagram: https://gitdiagram.com/caaaaa2562/pp3-api-catalogo

PP3 - API de Pedidos
Repositorio: https://github.com/ignaciohernandez/pp3-api-pedidos
API: https://pp3-api-pedidos-1.onrender.com/
Estado del servicio: https://pp3-api-pedidos-1.onrender.com/health
GitDiagram: https://gitdiagram.com/ignaciohernandez/pp3-api-pedidos